:root[data-theme="dark"]{--bg:#16181c;--ink:#e9e7e2;--muted:#a3a9b5;--accent:#5cc3c3;--card:#1f2228;--line:#30343c;--zh:#e0b277;--warn:#ff8a7a;--hl:#4a4020;color-scheme:dark} pre.d{font-family:Menlo,Consolas,"PingFang SC",monospace;font-size:13px;line-height:1.45;white-space:pre;overflow-x:auto} .say{background:var(--card);border-left:4px solid var(--accent);border-radius:6px;padding:10px 14px;margin:10px 0} .say .en{font-size:18px} .fu{margin:6px 0 6px 10px;font-size:16px} .fu b{color:var(--accent)} .key{background:var(--hl);border-radius:8px;padding:10px 14px;margin:10px 0} @media print{pre.d{white-space:pre-wrap;font-size:11px}body{font-size:15px}.card,.say{break-inside:avoid}}

后两小时 · 精简版（Bruce 10—11 · Kyle 11—12）

只留会被问到的东西。Bruce：8 道押题，每题几条思路 + 一段英文。Kyle：8 个现场改代码的任务 + 系统设计看 系统设计速成 第 0 节的 4 步。

10—11 点 · Bruce：数据感觉和建模（8 道押题）

B1 「我们的数据来自网上的职业档案，有什么偏差？怎么纠正？」

思路：

1. 覆盖偏差：白领、科技、年轻人、美国更容易有档案；蓝领、小企业少。
   Coverage bias: white-collar, tech, younger and US workers are over-represented.
2. 更新滞后：很多人换了工作很久才更新档案。
   Reporting lag: people update profiles months after a change.
3. 纠正：按官方统计（比如美国劳工统计局 BLS 的职业 × 地区就业人数）做重新加权，也叫事后分层（post-stratification）。
   Reweight to official statistics by occupation and region.
4. 验证：拿上市公司年报里披露的员工人数对一对。
   Validate against headcounts in public filings.

Online profiles over-represent white-collar, tech and younger workers, and people update them late. I'd correct coverage by reweighting each

occupation-by-region cell to official employment counts like BLS, and I'd handle lag separately with a nowcasting model. Then I'd validate the weighted headcounts against companies that report employee counts in their filings, and track that error over time.

追问「权重太大怎么办」：某些格子样本很少，权重会特别大、噪声大，要截断权重或把相邻格子合并。

B2 「档案更新滞后，怎么估计这个月的真实离职率 / 人数？」

思路（比方：快递还在路上）：上个月发生的跳槽，现在只有一部分人更新了，剩下的「还在路上」。用历史数据算出「事情发生后第 k 个月有多少比例的人已经更新」，再把最近几个月观测到的数按这个比例放大。这叫 nowcasting。

```
历史上：事件发生后 1 个月内更新 40%，3 个月内 75%，12 个月内 95%
这个月观测到 400 次跳槽（发生在上个月）→ 估计真实约 400 ÷ 0.40 = 1,000
```

Recent changes are under-reported because people update late. From history I'd estimate the reporting-delay curve, what share of moves shows up after one, three, twelve months, and scale recent counts up by it. If 40 percent show up in the first month and we see 400 moves, the estimate is about 1,000. Then I'd revise the number as more data arrives and measure how much past estimates were revised.

接你作业：你处理迟到快照的做法（只重放那个人）就是「数据晚到要能修正过去的结果」。

B3 「很多职位没写薪资，怎么预测？」

1. 标签：有薪资透明法的州（纽约、加州、科罗拉多）发布的岗位写了薪资范围，拿来当训练数据。
2. 特征：标准化后的职位、资历级别、地区、公司、行业、公司规模。
3. 目标取对数（log salary），因为薪资是**右偏的**（right-skewed）：大多数人 5 万到 15 万，少数人 100 万、300 万，画出来的分布左边堆成一座山、右边拖一条长尾

巴。直接预测原始数字，模型会被那几个超高薪的拉偏，而且「预测 5 万错 1 万」和「预测 50 万错 1 万」意义完全不一样。取对数后：① 分布变得接近正态，模型好学；② 误差变成百分比，5 万错 10% 和 50 万错 10% 一样重；③ 预测出来再 exp 回去。Salaries are right-skewed, a long tail of very high pay, so I model log salary: the distribution becomes roughly normal and errors become percentage errors, then I exponentiate back.

4. 先做简单基线：按「职位 × 地区」取中位数（median，不用平均数，平均数也会被高薪拉偏）。基线的作用是知道「不用模型能到多少」，模型只有比它好才值得。
5. 再上**梯度提升树（gradient boosting，如 XGBoost / LightGBM）**。为什么是它：① 特征全是类别型（职位、地区、公司、行业）+ 少量数值（资历、公司规模），树模型天然处理类别和非线性，不用手动做特征交叉（比如「纽约的软件工程师」和「纽约的护士」薪资差距不是简单相加）；② 表格数据上它几乎总是最强的基线，比线性回归准得多，又不像深度学习那样要大数据和调参；③ 训练快、能输出特征重要性，方便解释。为什么不用线性回归：假设特征效果线性相加，职位 × 地区的交互抓不住。为什么不用神经网络：数据只有几万到几十万条、特征是表格，神经网络没优势还难解释。Gradient boosting is the strongest default for tabular data with categorical features: trees capture interactions like title-by-location without hand engineering, train fast, and give feature importance. Linear models miss the interactions, and neural nets need more data for no gain here.
6. 评估：对数空间的 MAE，换算成百分比误差；按职位和地区分组看误差。
7. 坑：写了薪资的岗位本身就有偏（集中在某些州、某些公司），要检查迁移到别的州时误差会不会变大。

I'd train on postings from pay-transparency states, predict log salary from standardized title, seniority, location, company and industry, start from a title-by-location median baseline, then try gradient boosting. I'd report error as a percentage by segment, and check selection bias, because postings that disclose pay come from certain states and companies, so I'd test how well it transfers to states without those laws.

B4 「怎么把一千万种原始职位名称映射到标准分类？」

1. 先规范化：大小写、标点、缩写（MLE → machine learning engineer）、多语言。你作业里的 title_key 和词表就是这一步。
2. 高频的几千个：人工或规则直接映射，覆盖大部分流量。
3. 长尾：用文本向量找最近的标准职位，置信度低的交给大模型判断，再低的进人工审核。
4. 评估：分层抽样，按频次高低分层看准确率；大模型当评委时要用人工样本校准。
5. **人工校准具体怎么做**（你 take-home 里做过的，照实说）：
    1. **人工先标一小批**：从判官要判的那批里抽 60 对，分层抽（每类 15 对），自己逐对标「同一个 / 不同 / 拿不准」。这 60 对就是「金标准」（gold labels）。
    2. **让判官判同一批**：同样的 60 对、同样的提示词，看它和人工一致多少。你的结果：52/60 一致，kappa 0.66。
    3. **算一致性要用 kappa，不用简单百分比**：如果 90% 的对本来就是「同一个」，判官闭眼全答「同一个」也有 90% 一致。kappa 把这种碰巧一致扣掉，0.6 以上算可靠，0.8 以上算很好。
    4. **看分歧集中在哪**：你的 8 对分歧全在「同雇主同开始却分开」那一层。结论：判官在别的层可信，在这一层不可信，这一层的判官结果要打折或者多用人工。
    5. **按分歧修提示词**：判官系统性判错的，改提示词里的定义（比如明确「升职算同一份工作」），再在金标准上重跑，看 kappa 有没有涨。
    6. **两个判官互相对照**：Claude 和 GPT 用同一提示词判同一批，一致 kappa 0.66；两个都说对的更可信，两个不一致的那部分送人工。
    7. **上线后持续**：每个月再标一小批新的，看判官有没有漂移。

    Calibration means hand-labelling a small stratified gold set, having the judge label the same pairs, and measuring agreement with kappa rather than raw accuracy so chance agreement is removed. Then I look at where disagreements cluster, trust the judge only on strata where it agrees with humans, fix the prompt where it's systematically wrong, and cross-check two judges, sending their disagreements to humans. In the take-home that was 60 pairs, 52 agreed, kappa 0.66, with every disagreement in one stratum.
6. 版本化：分类表更新时要能回算历史数据，不然时间序列会断。

Normalize first: case, punctuation, abbreviations and languages, which is what my title key and glossary did. Map the head of the distribution with rules and

review, use embeddings to find the nearest standard title for the long tail, send low-confidence cases to an LLM, and the lowest to humans. Evaluate with stratified samples by frequency, calibrate the LLM judge against human labels, and version the taxonomy so history can be recomputed.

B5 「没有标签，线上怎么知道你的合并结果质量还好？」

直接讲你作业的做法：

- 不需要标签的检查：覆盖率（99.94%）、倒序日期（0）、可疑拆分数（同人同快照同雇主同起始却两个 id，267k → 37k）。每天算，设报警。
- 定期抽样：按决定类型分层，每层抽 75 个，两个大模型评委加少量人工。
- 漂移监控：每天新开 id 的比例、合并比例、每人平均职位数，突然变了就查。

Three layers. Label-free checks every run, like coverage, inverted dates and suspicious splits, with alerts. Periodic stratified samples judged by two LLMs and a few human labels. And drift monitoring on the share of new ids, merges and positions per person, so a scraper change shows up as a jump before anyone reads the data.

B6 「某个指标这个月突然涨了 20%，你怎么查？」

1. 先怀疑数据，后相信世界：上游抓取量、抓取器版本、去重逻辑、分类表版本有没有变。
2. 拆分：按地区、行业、公司、数据源往下拆，看涨幅是不是集中在一块。
3. 对照外部：同期的官方数据、新闻（比如某公司大裁员）。
4. 确认是数据问题就修，并加一条回归检查；是真的就写清楚原因。
5. **为什么修完还要加一条回归检查（regression check）**：修 bug 只能保证「这次好了」，不能保证「以后不会再坏」。比方：家里水管漏了，修好之外再装一个漏水报警器。回归检查就是那个报警器：把这次的错误写成一条自动检查，以后每次跑完都验一遍，同样的错再出现就立刻报警，而不是等客户发现。具体两种：① **回归测**

试：把出错的那个真实案例写进测试集，比如 Veeva 那个人的降级抓取，以后谁改代码改坏了，测试直接挂；② **数据断言**：把症状写成指标规则，比如「某指标单月涨幅超过 20% 就告警」「倒序日期必须为 0」，每次运行自动检查。你 take-home 里 51 个测试中约 20 个就是这么来的：判官每抓到一个错，修完就加一个用真实数据写的回归测试。A fix only proves it works today; a regression check makes sure the same failure can't come back silently. I turn the real failing case into a test, and the symptom into a data assertion that runs every day. About twenty of my tests came from judge findings that way.

I'd assume a data issue first. Did source volume, the scraper, dedup logic or the taxonomy version change? Then I'd slice by region, industry, company and source to see if the jump is concentrated. I'd compare with outside signals like official statistics or news. If it's a data bug, fix it and add a regression check, which is how I handled every judge finding in my take-home.

B7 「从职位名称推断资历级别（seniority）怎么做？」

1. 规则先行：intern、junior、senior、staff、principal、director、VP、C 级，各语言对应词。
2. 没有关键词的：用工作年限（第一份工作到现在）、团队规模、公司规模辅助，训练一个分类器。
3. 坑：不同公司的头衔通胀不一样（创业公司的 VP 可能只是第三个员工）。要按公司规模校准。

Start with keyword rules across languages, then for titles without seniority words use years since the first job and company size in a classifier. Watch for title inflation: a VP at a ten-person startup isn't a VP at a bank, so calibrate by company size.

B8 万能收尾句

My general approach is the same as in the take-home: define what "right" means, build the simplest baseline, measure it on stratified samples, and only

add complexity where the errors are.

先定义什么叫对，做最简单的基线，分层抽样衡量，只在错误集中的地方加复杂度。

11–12 点 · Kyle：写代码 + 系统设计

- **写代码**：先说思路 → 写测试 → 改代码 → 跑测试，边做边说。下面 8 个任务是他最可能让你当场加的。
- **系统设计**：问需求 → 估算 → 画高层图 → 深挖一个点。框架和素材在 系统设计速成，题目八成是「每天几亿条档案的去重流水线」。

11–12 点 · Kyle：现场改代码（8 个可能的任务）

做法固定 5 步：① 用一句话复述需求 ② 说在哪个文件改 ③ 先写一个会失败的测试 ④ 改代码 ⑤ 跑 `uv run pytest`，说说可能影响到哪些别的规则。

E1. 公司缩写没网址（RL / Revelio Labs）也要认出来

改哪：match.py 的 is_candidate 兜底那一段。normalize.py 里已经有 `_initials`。

```
from .normalize import _initials

def _initials_match(a: str, b: str) -> bool:
    """'rl' vs 'revelio labs': a short key equal to the other's initials."""
    ca, cb = a.replace(" ", ""), b.replace(" ", "")
    return ((len(ca) <= 5 and len(b.split()) >= 2 and ca == _initials(b))
            or (len(cb) <= 5 and len(a.split()) >= 2 and cb == _initials(a)))

# in is_candidate, before the final rename fallback:
ck = company_key(row["company_raw"])
if ck and any(_initials_match(ck, k) for k in pos.company_keys) \
        and start_compatible(row["startdate"], pos.startdate):
    return True
```

测试（照 tests/test_match.py 的写法）：

```
def test_company_initials_without_url_keep_one_id():
    from coalesce.step import step
    st = UserState(9)
    step(st, D(2026, 9, 1), [_row("Revelio Labs", "Machine Learning Engineer", D(2026,
    step(st, D(2026, 9, 16), [_row("RL", "MLE", D(2026, 8, 1))])
    assert len(st.positions) == 1
```

要主动说的风险：短缩写会撞（AB 可能是很多公司）。所以要求开始日期兼容，而且只在这个人自己的经历里比，范围很小。上线前用判官抽样看新合并的精确率。

E2. 旧经历没有结束日期，也要识别再入职

思路：现在规则 3 只看「已知结束日期」。可以加：同一次快照里同时出现两行、同雇主同职位、开始日期相差很远 → 两条（简历上本来就是两行）。不同快照之间就难了：开始日期从 2015 变成 2022，可能是改日期，也可能是回来了。说法："Across snapshots I can't tell an edited start date from a rehire without an end date, so I'd only split when both stints appear in the same snapshot."

E3. 把贪心换成最优配对

先给他一个贪心会错的例子：

```
          P0   P1
   r0     60   58
   r1     57   --  (不是候选)
贪心: r0->P0 (60). r1 只能要 P0, 已被占 -> r1 开新编号. 配上 1 对.
最优: r0->P1 (58) + r1->P0 (57) = 115 > 60. 配上 2 对.
```

```
import numpy as np
from scipy.optimize import linear_sum_assignment

def optimal_pairs(rows, cands):  # cands: {row_index: [Position, ...]}
    pos = list({p.position_id: p for ps in cands.values() for p in ps}.values())
    idx = list(cands)
    s = np.full((len(idx), len(pos)), -1.0)
    for a, i in enumerate(idx):
        for p in cands[i]:
            s[a, pos.index(p)] = _score(rows[i], p)
```

```
    r, c = linear_sum_assignment(-s)          # maximize total score
    return {idx[a]: pos[b] for a, b in zip(r, c) if s[a, b] >= AMBIGUOUS_MIN_SCORE}
```

说：只对「有争抢」的那部分用；量一下翻转了多少个决定，抽样给判官看。

E4. 用输出算「每家公司每月在职人数」

关键：用每个人最新的 asof_ts 那一组行（否则同一段经历会被数很多次）。如果是给客户做回测，要用「当时」的 asof（时间点正确）。

```
-- DuckDB
WITH latest AS (
  SELECT user_id, MAX(asof_ts) AS ts FROM out GROUP BY user_id),
cur AS (
  SELECT o.* FROM out o JOIN latest l
    ON o.user_id = l.user_id AND o.asof_ts = l.ts),
months AS (
  SELECT CAST(m AS DATE) AS m
  FROM generate_series(DATE '2020-01-01', DATE '2026-09-01', INTERVAL 1 MONTH) t(m))
SELECT company_raw, m, COUNT(DISTINCT user_id) AS headcount
FROM cur JOIN months
  ON cur.startdate <= m AND (cur.enddate IS NULL OR cur.enddate >= m)
GROUP BY company_raw, m
ORDER BY company_raw, m;
```

要提：company_raw 还没做公司实体消歧，真实场景应该先映射成公司 id（第 2 章 c 题）。

E5. 加「被遗忘权」删除名单

思路：一张 (user_id) 或 (position_id) 的删除表。在 runner 写输出前过滤掉；状态里也删，免得第二天又带回来。历史表里也要删，不然重放时会复活。说："It has to be applied in three places: output, state and history. Otherwise a replay brings it back."

E6. 迟到数据重放后编号保持不变

- A：重放得到新的 positions 后，按别名重叠和开始日期，和旧 positions 一一对应，沿用旧编号；对不上的才发新号（序号从旧的 next_seq 往后接）。

- B：不重放，把迟到的快照当成「补丁」，只补空字段、不改配对。快，但不完全等于全量。
- C：照旧重放，但给数据加版本号，下游按版本读。

推荐 A。说清代价：A 的对应关系本身也是一个小配对问题。

E7. 把学到的打分器接进去

思路：在 _score 那里换成模型的概率，阈值按「精确率 ⩾ 97%」选（之前的数：0.8 阈值时精确率 96.7%、召回 79.0%）。先只在「有争抢」那一步用，跑 A/B 对比脚本（eval/ab_semantic.py 就是干这个的），新合并和新拆开都抽样给判官。

E8. 让它更快

- 先量：哪个阶段最慢（分桶、清洗、逐人循环、写文件）。runner 已经记了分桶耗时和每个桶的秒数。
- 候选查找现在是「行 × position」两两比。人多经历多时，可以先按 company_key 建个小字典。
- 相似度函数已经有 lru_cache。可以把 distinct 职位名对的相似度先批量算好。
- 大规模时瓶颈在读写：减少中间落盘，比如分桶直接在 Spark shuffle 里做。

任何人问「弱点」时：自己先说

被问「有什么问题」时，主动说出 2–3 条，比被他挑出来强得多。

1. **同公司同开始日期却分开的那层**：大多是错的（Claude 38% 对，GPT 8% 对）。但只占 0.26% 的编号。下一步：多标注这一层，或者放宽快照内去重。
2. **同起止日期的两份真实并行工作会被合掉**：约 25%。例：Author / Psychologist。这是故意选的那一边的错。
3. **旧的那段没有结束日期时，后面同职位的再入职会被合进去**。因为规则 3 只在知道结束日期时生效。
4. **公司模板描述**：同公司两个不同角色用了一样的公司介绍，可能被合掉（Caterpillar 例子）。
5. **词典只覆盖常见词**。

6. **缩写的公司名 + 没网址 + 职位也改了 → 会变成两个编号**。例：「Revelio Labs, Machine Learning Engineer」变成「RL, MLE」且没有网址。这条 README 没写，是今晚用你的代码跑出来的。题目里的 Case B 能过，是因为两行网址一样。被问「没有网址怎么办」就说这条，再说 1.17 的修法 E1。
7. **迟到数据重放后，这个人的编号可能变**。已经写进 restated 让下游知道。
8. **阈值是手定的**（55、120、62 天、+25），用评估集核过，但没有系统调参。
9. **贪心不是最优配对**（1.17 E3 有例子）。快照里争抢的行很少，所以影响小，但没量过。
10. **Spark 没上真集群**；历史表每次整份重写，生产应该按用户分区、只追加。